

In-Context Learning for Finite-Length Markov Chains

Swathi Narashiman
Summer@EPFL 2025 Intern



May 22, 2026

What is In-Context Learning?

In-context learning is a method where LLMs learn to perform tasks by observing examples within the input prompt, rather than through explicit training on labeled data

Why Markov Chains ?

Markov chains are used in ICL to model and understand how these models learn to predict the next element in a sequence based on the preceding context.

Core Question : What estimators do the transformers implement for finite length Markov chains ?

It is a 2 Layer Attention circuit that helps complete patterns in a Large language model.

Layer 1: Copies information from previous token to the next token

Layer 2: Performs prefix matching i.e. counts the number of tokens in context that was preceded by the current token

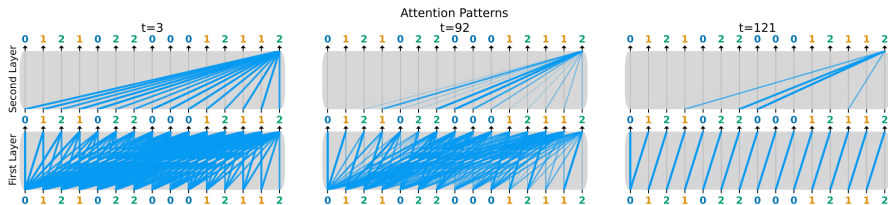


Figure: Induction Head

The Evolution of Statistical Induction Heads: [link](#)

- Analysis of transformers trained on sequences from Markov chains with Dirichlet priors.
- Transformer learns to implement a counter stage-wise

Uniform $\implies$ *Unigram* $\implies$ *Bigram*

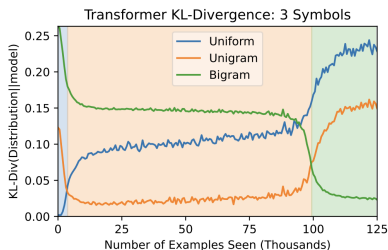


Figure: In-Context Learning Dynamics

- Each sequence is generated from a first-order Markov Chain.
- The transition matrix for each sequence is sampled from a Dirichlet prior:

$$P_{i,:} \sim \text{Dirichlet}(\kappa^*)$$

- The stationary distribution of the markov chain is denoted by $\pi \in R^{|V|}$
- Initial tokens are sampled uniformly from the vocabulary.
- For $t > 0$, token x_t is sampled using:

$$x_t \sim \text{Categorical}(P_{x_{t-1},:})$$

- No explicit access to P — model must infer transition matrix from context.

Smoothed Bigram Counter Assume true TPM P has $|V|$ states and rows sampled as:

$$P_i \sim \text{Dirichlet}(\kappa^*)$$

- Given context x_1^T , compute bigram counts N_{im}, N_i (how often $i \rightarrow m$ occurs).

$$N_{im} = \#\{t : x_{t-1} = i, x_t = m\}, \quad N_i = \#\{t : x_{t-1} = i\}$$

- Posterior distribution:

$$\mathbb{P}(P_i \mid x_1^T) = \text{Dir}(\kappa_1^* + N_{i1}, \dots, \kappa_{|V|}^* + N_{i|V|})$$

- Marginal for each transition:

$$\mathbb{P}(P_{im} \mid x_1^T) = \text{Beta}(\kappa_m^* + N_{im}, \sum_j \kappa_j^* + N_i - \kappa_m^* - N_{im})$$

- Posterior mean (smoothed bigram estimate):

$$\mathbb{E}[P_{im} \mid x_1^T] = \frac{\kappa_m^* + N_{im}}{\sum_j \kappa_j^* + N_i}$$

- **Disentangled Multi-Head Self Attention**

The output from each head is concatenated, hence **traceable**.

- The hidden size of the model grows as follows,
Assume at layer l , we have m_l heads. And let the dimensions be $d_0, d_1, \dots, d_L$.

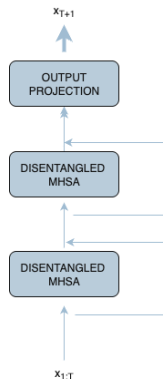
$$d_l = (1 + m_l)d_{l-1}$$

- **Relative Positional Encoding**

Define weight matrix:

$$RPE(i, j) = \begin{cases} w_{j-i}, & i > j, |i-j| \leq T \\ -\infty, & \text{otherwise} \end{cases}$$

Note: We disabled the LayerNorm and the Bias in the Output projection layer as we experimentally found that they had no effect in ICL



Let $x_1^T = [x_1, x_2, \dots, x_T]$ be the one-hot encoded input sequence $(x_1^T \in \mathbb{R}^{|V| \times T})$

Layer 1

$$A^{(1)} = x_1^{T\top} W_a^{(1)} x_1^T \quad (A^{(1)} \in \mathbb{R}^{T \times T}, W_a^{(1)} \in \mathbb{R}^{|V| \times |V|})$$

Addition of RPE

$$\hat{S}_{t,i}^{(1)} = A_{t,i}^{(1)} + RPE_{t,i}^{(1)}$$

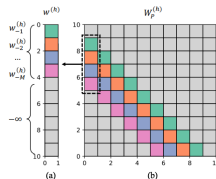
$$S_{t,:}^{(1)} = \text{Softmax}(\hat{S}_{t,:}^{(1)})$$

Output

$$z_t^{(1)} = \sum_i x_i S_{t,i}^{(1)} \quad (z_t^{(1)} \in \mathbb{R}^{|V|}, h_t^{(1)} \in \mathbb{R}^{2|V|})$$

After Residual Connection

$$h_t^{(1)} = \begin{bmatrix} x_t \\ z_t^{(1)} \end{bmatrix}$$



Layer 2

$$A^{(2)} = h_T^{(1)\top} W_a^{(2)} h_T^{(1)} \quad (W_a^{(2)} \in \mathbb{R}^{2|V| \times 2|V|})$$

Addition of RPE

$$\hat{S}_{i,j}^{(2)} = A_{i,j}^{(2)} + RPE_{i,j}^{(2)}$$

$$S_{i,:}^{(2)} = \text{Softmax}(\hat{S}_{i,:}^{(2)})$$

Output

$$z_t^{(2)} = \sum_i h_t^{(i)} S_{t,i}^{(2)} \quad (z_t^{(2)} \in \mathbb{R}^{2|V|})$$

After Residual Connection

$$h_t^{(2)} = \begin{bmatrix} h_t^{(1)} \\ z_t^{(2)} \end{bmatrix} \quad (h_t^{(2)} \in \mathbb{R}^{4|V|})$$

Output Projection

$$\mathcal{T}_T = W_o^\top h_t^{(2)} \quad (\mathcal{T}_T \in \mathbb{R}^{|V|})$$

- The parameter θ consists of 5 trainable matrices
 - $W_a^{(1)}, W_a^{(2)}$ — Attention matrix for 2 layers
 - $RPE^{(1)}, RPE^{(2)}$ — Positional encoding for 2 layers
 - W_o — The output projection matrix
- We train the First Variant of the model on Cross Entropy Loss

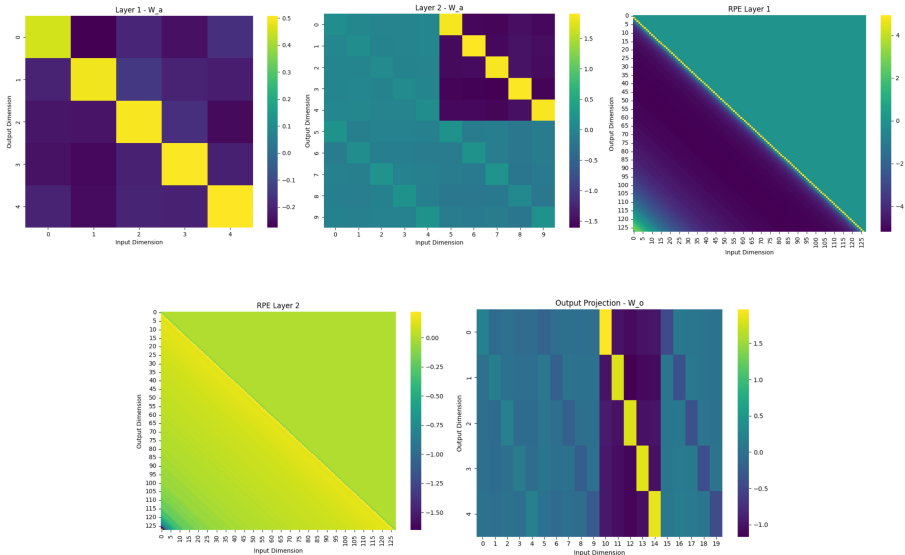
$$\mathcal{L}_{CE}(\theta) = \mathbb{E}_{P \sim \text{Dirichlet}(\kappa^*)} \mathbb{E}_{x_1^{T+1} \sim P} \left[-\log (\text{Softmax}(\mathcal{T}_T)[x_{T+1}]) \right]$$

- A Second Variant was trained on NLL Loss by fixing W_o to Identity appropriately so as to output the probabilities directly

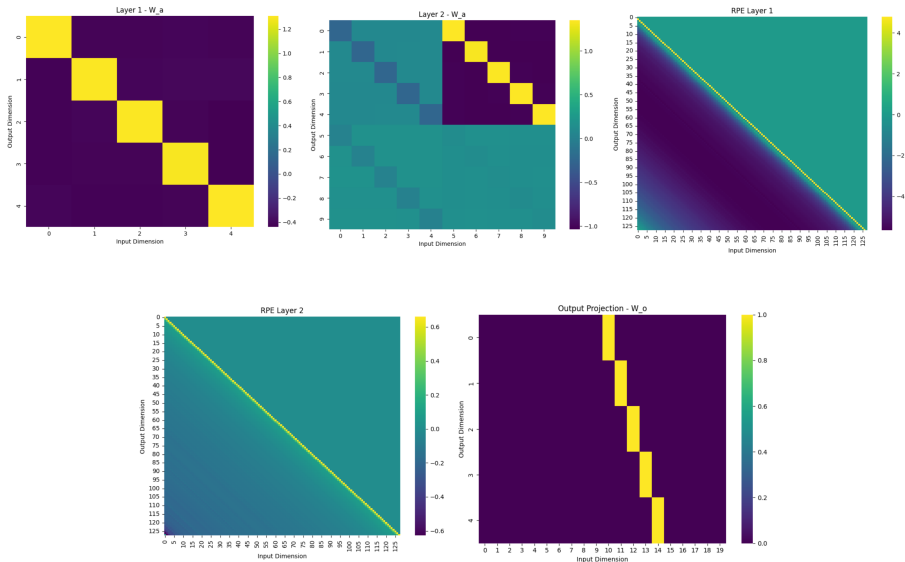
$$\mathcal{L}_{NLL}(\theta) = \mathbb{E}_{P \sim \text{Dirichlet}(\kappa^*)} \mathbb{E}_{x_1^{T+1} \sim P} \left[-\log (\mathcal{T}_T[x_{T+1}]) \right]$$

- We train the models for 1M iterations for order 1 markov chains of length $T = 128$ and vocab size $|V| = 5$ with each layer having 1 head.

Heatmaps of matrices - Cross Entropy



Heatmaps of matrices - NLL



Based on the heatmaps observed in the previous slide, we fix the matrices as follows:

- **Frozen Induction Head trained on Cross Entropy Loss:**

3 learnable parameters - α, β, γ ($\delta = -10^9$)

Layer 1:

$$W_a^{(1)} = \mathbf{0}_{|V| \times |V|} \quad , \quad RPE^{(1)} = \alpha \begin{bmatrix} -1 & 0 & 0 & \cdots & 0 \\ 1 & -1 & 0 & \cdots & 0 \\ -1 & 1 & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & -1 & \cdots & -1 \end{bmatrix}$$

Layer 2:

$$W_a^{(2)} = \beta \begin{bmatrix} \mathbf{0}_{|V| \times |V|} & \mathbf{I}_{|V| \times |V|} \\ \mathbf{0}_{|V| \times |V|} & \mathbf{0}_{|V| \times |V|} \end{bmatrix} \quad , \quad RPE^{(2)} = \delta \begin{bmatrix} 0 & \cdots & 0 \\ \vdots & \ddots & 0 \\ 1 & \cdots & 0 \end{bmatrix}$$

$$W_o = \gamma \begin{bmatrix} \mathbf{0}_{|V| \times |V|} & \mathbf{0}_{|V| \times |V|} & \mathbf{I}_{|V| \times |V|} & \mathbf{0}_{|V| \times |V|} \end{bmatrix}$$

- **Frozen Induction Head with NLL Loss:** 2 learnable parameters α, β , $\gamma = 1$.

Plot the KL Divergence between Ground Truth TPM and Model's predicted probabilities

- This is compared against:
 - **Bigram estimator (no smoothing)** $\sim \frac{N_{im}}{N_i}$
 - **Optimal smoothed bigram estimator** $\sim \frac{\kappa_m^* + N_{im}}{\sum_j \kappa_j^* + N_i}$
- **Observation:** KL divergence for the **Frozen Induction Head** closely matches that of the **Fully Trained Model**, for both Cross Entropy and NLL Loss
- This suggests the frozen parameterization effectively captures the transition structure in the data

KL_vs_Time

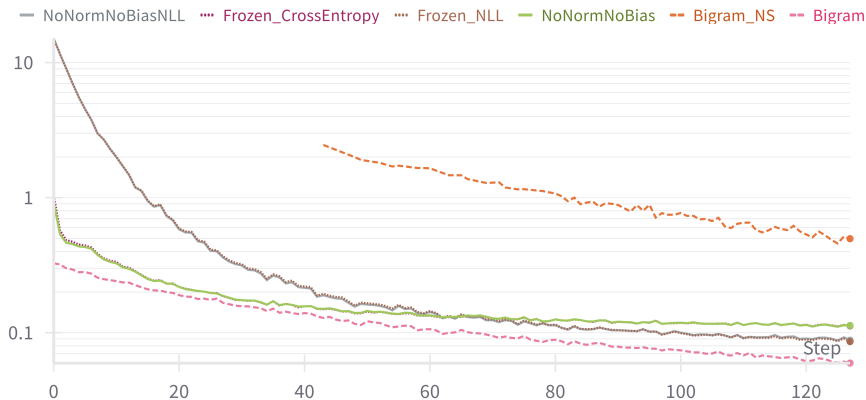


Figure: KL Divergence (log scale) vs T

Layer 1 Representation (Copying): For large α , the residual stream concatenates both the current and previous token:

$$h_t^{(1)} = \begin{bmatrix} x_t \\ x_{t-1} \end{bmatrix} \in \mathbb{R}^{2|V|}$$

Here, x_t is a one-hot vector for token x_t .

Layer 2 Attention Scores: We construct an attention matrix where the attention weight from x_T to position t is:

$$\hat{S}_{T,t} = h_T^{(1)\top} W_a^{(2)} h_t^{(1)} = \beta \cdot \mathbb{I}\{x_T = x_{t-1}\}$$

Softmax Attention Weights:

$$S_{T,t}^{(2)} = \frac{e^{\beta \cdot \mathbb{I}\{x_T = x_{t-1}\}}}{\sum_{u=1}^T e^{\beta \cdot \mathbb{I}\{x_T = x_{u-1}\}}}$$

Interpretation: As $\beta \rightarrow \infty$, we get the bigram estimator.

Attention Output Vector at T :

$$z_T^{(2)} = \sum_{t=1}^T S_{T,t}^{(2)} \cdot h_t^{(1)} = \sum_{t=1}^T S_{T,t}^{(2)} \cdot \begin{bmatrix} x_t \\ x_{t-1} \end{bmatrix}$$
$$z_T^{(2)}[m] = \sum_{t=1}^T S_{T,t}^{(2)} \cdot \mathbb{I}\{x_t = m\} \quad \text{for all } m \leq |V|$$

Define:

- $N_{im} = \#\{t \in [2, T] : x_{t-1} = i, x_t = m\}$
- $K_i = \#\{t \in [2, T] : x_t = i\}$
- $\bar{N}_{im} = \#\{t \in [2, T] : x_{t-1} \neq i, x_t = m\} = K_m - N_{im}$
- $N_i = \#\{t \in [2, T] : x_{t-1} = i\}$
- $\bar{N}_i = \#\{t \in [2, T] : x_{t-1} \neq i\} = T - 1 - N_i$

So:

$$z_T^{(2)}[m] = \frac{N_{im} e^\beta + \bar{N}_{im}}{N_m e^\beta + \bar{N}_i} = \frac{N_{im} + \bar{N}_{im} e^{-\beta}}{N_i + \bar{N}_i e^{-\beta}}.$$

Smoothed Estimate from Transformer $\bar{N}_{im} = K_m - N_{im}$, $\bar{N}_i = T - 1 - N_i$

$$z_T^{(2)}[m] = \frac{N_{im} + \bar{N}_{im} e^{-\beta}}{N_i + \bar{N}_i e^{-\beta}} = \frac{(1 - e^{-\beta})N_{im} + e^{-\beta} K_m}{(1 - e^{-\beta})N_i + (T - 1)} = \frac{N_{im} + \frac{e^{-\beta}}{1 - e^{-\beta}} K_m}{N_i + \frac{e^{-\beta}}{1 - e^{-\beta}} (T - 1)}$$

Unigram Belief

$$\mathbb{E}[P_m \mid x_1^T] = \frac{|\{t \in [2, T] : x_t = m\}|}{T - 1} = \frac{K_m}{T - 1}$$

Bigram (No Smoothing) : $\kappa_i = 0 \quad \forall i$

$$\mathbb{E}[P_{im} \mid x_1^T] \sim \frac{N_{im}}{N_i}$$

Interpretation: The transformer implicitly blends bigram and unigram statistics — a **smoothed counter** with temperature β controlling interpolation.

Gradient: Let $\eta = \frac{e^{-\beta}}{1 - e^{-\beta}}$ and take 1st-order approx. around $\mathbb{E}[K_m] = \frac{T-1}{|V|}$

$$\begin{aligned}\nabla_{\eta} \mathbb{E} \left[-\log z_T^{(2)}[m] \right] &= \mathbb{E} \left[-\frac{K_m}{N_{im} + \eta K_m} + \frac{T-1}{N_i + \eta(T-1)} \right] \\ &\approx \mathbb{E} \left[-\frac{(T-1)/|V|}{N_{im} + \eta(T-1)/|V|} + \frac{T-1}{N_i + \eta(T-1)} \right].\end{aligned}$$

Smoothed Bigram : $\kappa_i = \kappa \quad \forall i$

$$\hat{p}(x_1^T; \kappa)[m] = \frac{N_{im} + \kappa}{N_i + |V|\kappa}$$

κ^* is the optimal level of smoothing, i.e., gradient of the loss is zero at $\kappa = \kappa^*$:

$$\nabla_{\kappa} \mathbb{E} \left[-\log \hat{p}(x_1^T; \kappa^*)[m] \right] = \mathbb{E} \left[-\frac{1}{N_{im} + \kappa^*} + \frac{|V|}{N_i + |V|\kappa^*} \right] = 0.$$

Conclusion : Setting $\eta = \frac{|V|\kappa^*}{T-1}$ leads to $\nabla_{\eta} \mathbb{E} \left[-\log z_T^{(2)}[m] \right] \approx 0!$

Setup:

- Fix $\gamma = 1$, $\kappa^* = 1$
- Train the model on sequences of various lengths: $T \in \{8, 16, 24, 32, \dots, 128\}$
- Learn optimal values of α and β that minimize the NLL
- Observe the dependence of β on T

Motivation: From the previous slide, we observed:

$$\frac{e^{-\beta^*}}{1 - e^{-\beta^*}} \cdot \frac{T - 1}{|V|} \approx \kappa^* \Rightarrow \beta^* \approx \ln \left(1 + \frac{T - 1}{|V|\kappa^*} \right)$$

By empirically training on increasing T , we test whether β^* indeed exhibits a logarithmic dependence on T , as predicted by the analytical derivation.

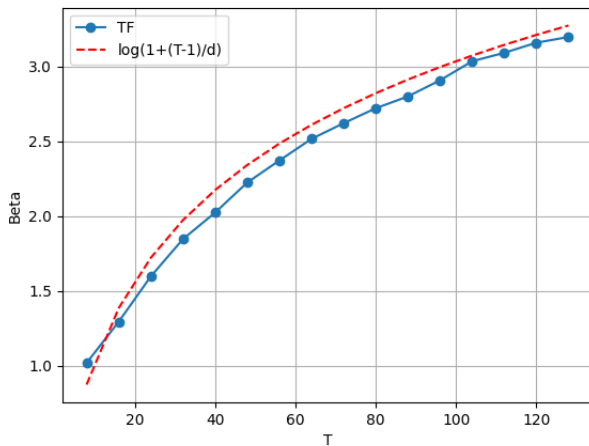
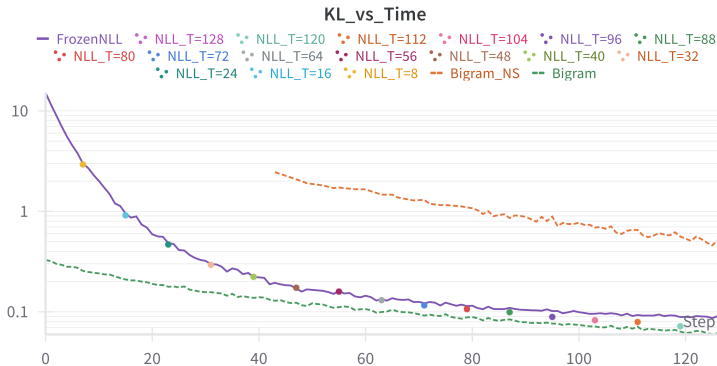


Figure: Optimal β^* vs T

Observation:

- As T increases, KL divergence tends to decrease.
- Indicates that the learned model gets closer to the ground truth for larger contexts.



- Transformers implement an estimator which appears to be in between the unigram and bigram for first order markov chains.
- The Transformer estimator converges quickly to the statistical best estimator (Smoothed bigram) than the Bigram without smoothing which converges as $T \rightarrow \infty$
- We can capture the Induction head for first order markov chains in 3 parameters - α, β, γ
- We observe that $\alpha \rightarrow \infty$ while training for optimal β , which means attention only on the $t - 1$ token in the first attention layer
- We obtain a closed form for beta which depends on T as

$$\beta^* \approx \ln \left(1 + \frac{T-1}{|V|\kappa^*} \right)$$

- Extending empirical evidence to Longer Markov chains ($T=256, 512$ etc.)
- Studying the estimator that is implemented with higher-order Markov Chains
- Derivation of optimal β and error
- Study β when the loss function is over all tokens and related construction
- Dynamics with α, β, γ parameterization
- How do MLPs affect the learning dynamics, the estimator and the performance, can we match the smoothed bigram?
- Study the same for absolute positional encoding
- Do we observe stage wise learning for higher order markov chains on the frozen transformer model ?
- what happens if δ is also learnt?

Thank You